Big Data Technologies – CCEE Study Cheatsheet

Sessions 4–11: HDFS & MapReduce Complete Guide

SESSION 4: HDFS Architecture Overview

Distributed File System

- Spreads files across multiple networked machines as a single logical file system
- Data split into fixed-size blocks (default 128 MB) and replicated across nodes
- Designed for high **throughput**, not low-latency random I/O

Core Components

- **NameNode** (master): Maintains file system namespace, block mappings, permissions; stores metadata (FsImage + EditLogs)
- **DataNode** (slave/worker): Stores actual data blocks; sends heartbeats and block reports to NameNode
- **Secondary NameNode**: NOT a backup; periodically merges FsImage and EditLogs for checkpointing

HDFS Daemons

1. NameNode daemon – manages metadata on master
2. DataNode daemon – stores blocks, sends heartbeat + block reports
3. Secondary NameNode daemon – checkpointing helper

Key Design Principles

- Write-once, read-many workload model
- Automatic re-replication on DataNode failure
- Rack-aware placement (replication factor = 3: 1 local rack, 2 other racks)

SESSION 5: HDFS Advanced Architecture

Replication & Fault Tolerance

- Default replication factor = 3
- Controlled per-file (not per-cluster)
- NameNode automatically triggers re-replication of under-replicated blocks
- More replicas = better availability + read throughput but higher storage/write cost

Rack Awareness

- HDFS tracks which rack each DataNode belongs to
- Places replicas across racks to reduce network traffic and improve fault tolerance
- Typical policy (RF=3): 1 replica on local rack, 2 on different racks

Data Pipelining (Write Path)

- Client → DN1 → DN2 → DN3 (chain formation)
- Acknowledgements flow back up the pipeline
- If DN fails mid-pipeline, pipeline reconstructs without failed node

Node Failure Management

- DataNodes send periodic heartbeats; absent heartbeats mark node as dead
- Under-replicated blocks are automatically re-replicated to other DataNodes
- Client-side retry on read failure (transparent)

HDFS High Availability (HA)

- Removes NameNode as single point of failure
- Active–Standby setup with shared edit logs (Quorum Journal Nodes or NFS)
- Standby continuously reads edits to stay synchronized
- Failover mechanism allows Standby to become Active quickly
- **≠ Secondary NameNode** (Secondary ≠ HA in non-HA clusters)

Scaling & Rebalancing

- Scale horizontally by adding DataNodes
- Rebalancing tool redistributes blocks for uniform disk usage
- Does not change replication factor; moves existing replicas

---

SESSION 6: Getting Started – Hadoop Setup

Operation Modes

1. **Standalone/Local** – Single JVM, no HDFS, local file system (debugging/testing)
2. **Pseudo-distributed** – All daemons on one machine, HDFS active (learning)
3. **Fully distributed** – Daemons spread across many machines (production)

Cluster Setup

- **Single-node**: Install Java + Hadoop on one VM/machine; configure core-site.xml, hdfs-site.xml, yarn-site.xml, mapred-site.xml; format NameNode
- **Multi-node**: Configure passwordless SSH; set up masters/slaves files; consistent config across all nodes; install Java + Hadoop on all

Remote Access

- PuTTY (Windows), macOS Terminal, Ubuntu Terminal – all use SSH for remote login
- Passwordless SSH required between master and workers for daemon management

Key Configuration Files

- **core-site.xml** – general Hadoop settings (e.g., fs.defaultFS)
- **hdfs-site.xml** – HDFS-specific (replication factor, NameNode/DataNode dirs)
- **yarn-site.xml, mapred-site.xml** – YARN and MapReduce configs

Monitoring & Maintenance

- Web UIs: NameNode UI (port 9870), YARN ResourceManager UI, HDFS health (live/dead nodes, capacity)
- Tools: Ambari, Cloudera Manager, Ganglia, Datadog
- Maintenance: balancing data, adding/removing nodes, checking block issues

Hadoop Benchmarks

- TeraSort, TestDFSIO, NNBench, MRBench – measure HDFS and MapReduce performance

Hadoop in Cloud

- AWS EMR, Azure HDInsight, GCP Dataproc – cloud-hosted Hadoop
- Easy scaling and pay-per-use; same logical architecture as on-premise

---

SESSION 7: Hadoop Architecture & Commands

Core Hadoop Modules (NOT ecosystem tools)

- **HDFS** – distributed file system (storage layer)
- **MapReduce** – parallel processing model (processing layer)
- **YARN** – resource management and job scheduling (management layer)
- **Hadoop Common** – shared utilities/libraries for all modules

Master–Slave Architecture

- NameNode + ResourceManager typically run on master
- DataNode + NodeManager run on worker nodes

HDFS Shell Commands (User-level)

- `hdfs dfs -ls /path` – list files
- `hdfs dfs -mkdir /dir`, `-mkdir -p` – create dirs (with parents)
- `hdfs dfs -put local.txt /hdfs/path` – upload
- `hdfs dfs -copyFromLocal local.txt /hdfs/path` – upload (alias)
- `hdfs dfs -get /hdfs/file local` – download
- `hdfs dfs -copyToLocal /hdfs/file local` – download (alias)
- `hdfs dfs -cat /hdfs/file` – display content
- `hdfs dfs -rm /file`, `-rm -r /dir` – delete

HDFS Admin Commands

- `hdfs dfsadmin -report` – cluster summary (capacity, used, live/dead DataNodes)
- `hdfs dfsadmin -safemode enter|leave` – control NameNode safe mode
- `hdfs dfsadmin -setQuota`, `-clrQuota` – directory quotas
- `hdfs fsck /path` – check file block health
- `hdfs dfsadmin -finalizeUpgrade`, `-rollingUpgrade`, `-saveNamespace` – advanced admin

HDFS Web Interface

- NameNode web UI (9870 or 50070): namespace tree, live/dead nodes, capacity, under-replicated blocks
- WebHDFS/HttpFS: HTTP/REST API to read, write, list HDFS files

---

SESSION 8: HDFS Data Storage & I/O

Write Path (Anatomy of File Write)

1. Client contacts **NameNode** with filename and replication factor
2. NameNode returns list of target DataNodes (rack-aware placement)

3. Client opens pipeline to DN1 → DN2 → DN3
4. Data sent in packets; acknowledgements flow back
5. Write succeeds once **minimum replicas** written; async replication fills remaining replicas
6. Blocks spilled to local disk on each DataNode

Read Path (Anatomy of File Read)

1. Client asks **NameNode** for block locations and DataNode addresses
2. NameNode returns sorted list of DataNodes (network-topology aware, closest first)
3. Client reads directly from nearest DataNode, then next block from next DataNode
4. On DataNode failure, client retries from another replica transparently
5. **DFSInputStream** hides block boundaries

Failure Handling

- **Write failure**: Pipeline reconstructs without failed DN; NameNode later re-replicates blocks
- **Read failure**: Client retries reading same block from another DataNode transparently
- Heartbeat + block reports enable NameNode to detect failed nodes and trigger re-replication

User vs Admin Commands

- **User commands** (`hdfs dfs ...`): file/directory operations (put, get, cat, rm, ls, mkdir)
- **Admin commands** (`hdfs dfsadmin ...`): cluster-level ops (report, safemode, quotas, fsck)

Web Interface & Monitoring

- NameNode web UI shows namespace, live/dead nodes, capacity, under-replicated blocks
- WebHDFS/HttpFS: HTTP API for HDFS access

SESSION 9: MapReduce Framework Basics

MapReduce Paradigm

Three phases:

1. **Map** – user-defined function transforms input records to intermediate key–value pairs (K2, V2)
2. **Shuffle & Sort** – framework groups all values by K2, merges, sorts keys, partitions to reducers
3. **Reduce** – user-defined function aggregates all values for each K2 into final (K3, V3)

Map and Reduce Tasks

- **Map task** – processes one input split; calls `map(key, value)` for each record; outputs to local disk
- **Reduce task** – receives all (K2, list) pairs for its partition; calls `reduce(key, Iterable<values>)`; writes final results to HDFS

MapReduce Execution Framework

- Client submits job → YARN ResourceManager allocates ApplicationMaster
- ApplicationMaster negotiates containers, starts Map/Reduce tasks on NodeManagers
- Tasks tracked and failed tasks re-executed
- **MRv1 legacy terms**: JobTracker (master job scheduler), TaskTracker (worker task executor)

Anatomy of Job Run

1. **Job submission** – client packages JAR, config, submits
2. **Input splits & mapping** – splits defined, map tasks created (ideally near data—data locality)
3. **Map output & shuffle** – mappers emit (K2, V2); partitioner decides reducer; spill to local disk
4. **Sort & reduce** – keys merged/sorted for each reducer; reducer processes groups
5. **Completion** – reducers write to HDFS (`part-r-*` files); job marked successful

Partitioner

- Decides **which reducer** receives a key–value pair
- Default: hash partitioner = `(hash(key) & Integer.MAX_VALUE) % numReducers`
- Same key always goes to same reducer (correctness requirement)

Combiner (Optional Mini-Reducer)

- Runs **locally on mapper output** before shuffle
- Reduces data transferred over network (pre-aggregation)
- **Must be associative & commutative** (like sum, max); cannot change keys
- Not guaranteed to run; logic must work if skipped entirely

Input Formats

- **InputSplit** – logical chunk assigned to one mapper (typically aligned with HDFS blocks)
- **Record** – unit processed by map(key, value); usually one line

Common InputFormats:

- **TextInputFormat** – default; key = byte offset, value = line text
- **KeyValueTextInputFormat** – splits line around separator (default tab) into key and value
- **SequenceFileInputFormat** – binary Hadoop key–value files
- **MultipleInputs** – allows multiple input paths with different InputFormats and mappers per path

Output Formats

- **TextOutputFormat** – writes key\tvalue per line (human-readable)
- **SequenceFileOutputFormat** – binary key–value storage (efficient)
- **MultipleOutputs** – write to multiple named output files from single job

Distributed Cache

- Distributes **read-only files** to all task nodes locally before execution
- Used for small lookup tables, config files, reference data
- Avoids re-reading same small file from HDFS per record
- Newer API: "Cache files/archives"

SESSION 10: MapReduce Programming in Java

Hadoop Writable Data Types

- Hadoop uses **Writable** interfaces for serialization (not raw Java primitives/String)
- **Common Writables**: IntWritable, LongWritable, FloatWritable, DoubleWritable, VIntWritable, VLongWritable, Text, BooleanWritable, ByteWritable, NullWritable
- Why: efficient binary serialization across network and disk

Java MapReduce Job Structure

Three main classes:

1. **Driver (Job Configuration)**
    - Sets job name, input/output paths, mapper/reducer classes
    - Configures key/value types, InputFormat/OutputFormat
    - Sets number of reducers, compression, combiner, partitioner
    - Submits via `job.waitForCompletion(true)`
2. **Mapper**
    - Extends `Mapper<KEYIN, VALUEIN, KEYOUT, VALUEOUT>`
    - Overrides `map(KEYIN key, VALUEIN value, Context context)`
    - Calls `context.write(keyOut, valueOut)` to emit intermediate pairs
3. **Reducer**
    - Extends `Reducer<KEYIN, VALUEIN, KEYOUT, VALUEOUT>`
    - Overrides `reduce(KEYIN key, Iterable<VALUEIN> values, Context context)`
    - Calls `context.write(keyOut, valueOut)` to emit final pairs

Map-only vs Reduce-only

- **Map-only** – set number of reducers to 0; ideal for per-record filtering/transformation with no cross-key aggregation
- **Reduce-only** – rare; use IdentityMapper + custom reducer; used when input already partitioned/grouped

Combiner & Partitioner Usage

- Configure combiner class if applicable; must be associative/commutative
- Configure custom partitioner for non-hash-based key distribution

Counters

- Built-in and user-defined numeric metrics (e.g., error counts, malformed records)
- Updated in map/reduce via `context.getCounter(...).increment(n)`
- Visible during job progress; useful for diagnostics

Job Schedulers

- **FIFO** – runs jobs in order
- **Capacity Scheduler** – fair allocation per queue
- **Fair Scheduler** – dynamic allocation based on demand
- Scheduler tries for **data locality** – place map tasks near HDFS blocks

Custom Writable

- Implement `Writable` (and `WritableComparable` if used as key)
- Define serialization via `write(DataOutput)` and `readFields(DataInput)`
- Used for composite keys or non-standard data types

Compression

- **Map output compression** – `mapreduce.map.output.compress` true; reduces shuffle data
- **Reducer output compression** – `mapreduce.output.fileoutputformat.compress` true; compresses final HDFS files
- **Codecs**: gzip, bzip2, Snappy, LZ4; often used with SequenceFile

---

SESSION 11: MapReduce Streaming

Hadoop Streaming Basics

- **Streaming utility** allows MapReduce in **any language** (Python, Perl, shell, etc.)
- Programs read from STDIN (input lines) and write to STDOUT (key\tvalue pairs)

- Hadoop handles splits, scheduling, shuffle, fault tolerance

Command pattern:

```
hadoop jar hadoop-streaming.jar \
  -input <HDFS_in> \
  -output <HDFS_out> \
  -mapper <mapper_cmd> \
  -reducer <reducer_cmd> \
  -file <script.py>
```

Streaming with Python

- Mapper: read lines from sys.stdin, parse, `print(key, value)`
- Reducer: read sorted (key, value) from sys.stdin, aggregate, `print(key, result)`
- Advantages: leverage Python libraries (PIL, scikit-learn, etc.) while using Hadoop's distributed engine

Complex MapReduce

- Non-text data processing (e.g., images, binary)
- Joins (co-groups, inner/outer joins via multiple inputs)
- Secondary sorting (composite keys, custom comparators)
- Multi-step pipelines (chained MapReduce jobs)

Image Processing with MapReduce

- Store images as **SequenceFiles** (key = image ID, value = binary image bytes)
- Mappers: decode, extract features, classify, or transform
- Reducers: aggregate statistics or results
- Use case: large batch processing (feature extraction, resize, classify)

Streaming Features

- Can use Java InputFormat/OutputFormat classes with non-Java mappers/reducers
- Still relies on same MapReduce execution framework (splits, shuffle, sort, tasks)
- Supports combiners, partitioners, counters just like Java MapReduce

---

QUICK REFERENCE: CCEE MCQ PATTERNS

HDFS Concepts

- "Single point of failure" → NameNode in non-HA
- "Write once, read many" → HDFS design principle
- "Replication factor = 3" → 1 local rack, 2 other racks (rack-aware)
- "Secondary NameNode" → NOT backup; does checkpointing only
- "Data locality" → map tasks placed near HDFS blocks
- "Heartbeat" → DataNode → NameNode; detects failure
- "Block report" → DataNode → NameNode; lists blocks held
- "Under-replicated" → blocks with fewer replicas than replication factor

MapReduce Concepts

- "Shuffle and sort" → groups values by key before reduce
- "Partitioner" → decides which reducer gets a key
- "Combiner" → optional mini-reducer on mapper output (must be associative/commutative)
- "Input split" → logical chunk per mapper task
- "Data locality" → schedule map tasks near input data
- "Writable" → serializable type for Hadoop I/O
- "Map-only" → reduce count = 0, no aggregation
- "Streaming" → MapReduce in any language via STDIN/STDOUT

Configuration & Operations

- `hdfs dfs` → user-level file/directory commands
- `hdfs dfsadmin` → admin-level cluster commands
- `hadoop jar hadoop-streaming.jar` → run Streaming jobs
- `-file <script>` → distribute script to nodes (Streaming)

Common Mistakes

- Confusing Secondary NameNode with HA Standby NameNode
- Assuming replication factor can be set per-cluster (it's per-file)
- Thinking reducer output always goes to HDFS (it does for MapReduce)
- Misunderstanding Combiner as guaranteed (it's optional)
- Using raw Java primitives instead of Writable types in MapReduce APIs

---

FORMULA & KEY NUMBERS

- **Default block size** – 128 MB (sometimes 256 MB)
- **Default replication factor** – 3
- **NameNode web UI port** – 9870 (or 50070 in older versions)
- **Rack-aware replica placement** – 1 local, 2 different racks
- **Heartbeat timeout** – ~10 minutes (node marked dead)
- **Partitioner formula** – `(hash(key) & Integer.MAX_VALUE) % numReducers`

SESSION-BY-SESSION SUMMARY

Session	Topic	Key Takeaway
4	HDFS Basics	NameNode (metadata), DataNode (data), Secondary NameNode (checkpointing)
5	HDFS Advanced	Replication, rack-awareness, write/read paths, HA, failure handling
6	Hadoop Setup	Modes (local, pseudo, fully distributed), config files, monitoring, cloud
7	Hadoop Architecture	HDFS/YARN/MapReduce/Common as core; shell commands; web UI
8	HDFS I/O	Write pipeline, read path, failure recovery, admin vs user commands
9	MapReduce Framework	Map/Shuffle/Reduce phases, splits, records, partitioner, combiner, cache
10	MapReduce Programming	Writables, Driver/Mapper/Reducer structure, map-only, counters, compression
11	Streaming	Hadoop Streaming (any language), Python MapReduce, image processing

FINAL EXAM TIPS (CCEE)

1. **No negative marking** – attempt all 40 questions per module
2. **Time management** – read carefully; 40 Qs ÷ 90 mins ≈ 2 min per Q
3. **Know definitions** – HDFS daemons, MapReduce phases, Writable types
4. **Understand architecture** – master–slave, pipeline, failure recovery
5. **Commands matter** – `hdfs dfs` vs `hdfs dfsadmin`, `-file` vs `-codec`
6. **Read carefully** – "True/False," multi-select, "which is NOT"
7. **Rack awareness** – default: 1 local, 2 remote (not all same rack)
8. **MapReduce logic** – Combiner is optional; Partitioner is required; Reducer group-by-key
9. **Streaming I/O** – STDIN/STDOUT, tab-separated, any language
10. **Review code**: Word count (mapper emits (word, 1), combiner sums locally, reducer sums globally)

End of Cheatsheet. Good luck on your CCEE exam!